BankDB.sql

```sql
CREATE TABLE Account (
    FOREIGN KEY (Customer_ID)
    REFERENCES Customer(Customer_ID)
);

CREATE TABLE Bank_Transaction (
    Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (Account_No)
    REFERENCES Account(Account_No)
);

CREATE TABLE Loan (
    Loan_ID INT PRIMARY KEY,
    Customer_ID INT,
    Loan_Type VARCHAR(30),
    Loan_Amount DECIMAL(12,2),
    Interest_Rate DECIMAL(5,2),

    FOREIGN KEY (Customer_ID)
    REFERENCES Customer(Customer_ID)
);

SHOW TABLES;
```

I/O | AI Agent | 176 ms

STDIN

Input for the program ( Optional )

Output:

```
+--------------------------------+
| Tables_in_sandbox_db |
+--------------------------------+
| account                        |
| bank_transaction               |
| customer                       |
| loan                           |
+--------------------------------+
```

Dark | Wiki | MySQL

BankDB.sql

```sql
 VALUES
 (10001, 101, 'Savings', 50000, 'Hyderabad'),
 (10002, 102, 'Savings', 75000, 'Vijayawada'),
 (10003, 103, 'Current', 120000, 'Bangalore'),
 (10004, 104, 'Savings', 45000, 'Chennai'),
 (10005, 105, 'Current', 90000, 'Hyderabad');

 INSERT INTO Bank_Transaction
 (Account_No, Transaction_Type, Amount)
 VALUES
 (10001, 'DEPOSIT', 10000),
 (10002, 'DEPOSIT', 15000),
 (10003, 'WITHDRAW', 20000),
 (10004, 'DEPOSIT', 5000),
 (10005, 'WITHDRAW', 10000);

 INSERT INTO Loan
 (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)
 VALUES
 (501, 101, 'Home Loan', 5000000, 7.5),
 (502, 102, 'Education Loan', 1000000, 6.5),
 (503, 103, 'Car Loan', 800000, 8.2),
 (504, 104, 'Personal Loan', 500000, 10.5);

 SELECT * FROM Customer;
 SELECT * FROM Account;
 SELECT * FROM Bank_Transaction;
 SELECT * FROM Loan;
```

I/O | AI Agent | 238 ms

STDIN

Input for the program ( Optional )

Output:

```
+-------------+---------------+------------+-----------------+------------+
| Customer_ID | Customer_Name | Phone      | Email           | City       |
+-------------+---------------+------------+-----------------+------------+
|         101 | Ravi Kumar    | 9876543210 | ravi@gmail.com  | Hyderabad  |
|         102 | Priya Sharma  | 9876543211 | priya@gmail.com | Vijayawada |
|         103 | Arjun Reddy   | 9876543212 | arjun@gmail.com | Bangalore  |
|         104 | Sneha Rao     | 9876543213 | sneha@gmail.com | Chennai    |
|         105 | Kiran Kumar   | 9876543214 | kiran@gmail.com | Hyderabad  |
+-------------+---------------+------------+-----------------+------------+
+------------+-------------+--------------+----------+------------+
| Account_No | Customer_ID | Account_Type | Balance  | Branch     |
+------------+-------------+--------------+----------+------------+
|      10001 |         101 | Savings      |  50000.00 | Hyderabad  |
|      10002 |         102 | Savings      |  75000.00 | Vijayawada |
|      10003 |         103 | Current      | 120000.00 | Bangalore  |
|      10004 |         104 | Savings      |  45000.00 | Chennai    |
|      10005 |         105 | Current      |  90000.00 | Hyderabad  |
+------------+-------------+--------------+----------+------------+
```

Dark | Wiki | MySQL

BankDB.sql

```sql
 VALUES
 (10001, 101, 'Savings', 50000, 'Hyderabad'),
 (10002, 102, 'Savings', 75000, 'Vijayawada'),
 (10003, 103, 'Current', 120000, 'Bangalore'),
 (10004, 104, 'Savings', 45000, 'Chennai'),
 (10005, 105, 'Current', 90000, 'Hyderabad');

 INSERT INTO Bank_Transaction
 (Account_No, Transaction_Type, Amount)
 VALUES
 (10001, 'DEPOSIT', 10000),
 (10002, 'DEPOSIT', 15000),
 (10003, 'WITHDRAW', 20000),
 (10004, 'DEPOSIT', 5000),
 (10005, 'WITHDRAW', 10000);

 INSERT INTO Loan
 (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)
 VALUES
 (501, 101, 'Home Loan', 5000000, 7.5),
 (502, 102, 'Education Loan', 1000000, 6.5),
 (503, 103, 'Car Loan', 800000, 8.2),
 (504, 104, 'Personal Loan', 500000, 10.5);

 SELECT * FROM Customer;
 SELECT * FROM Account;
 SELECT * FROM Bank_Transaction;
 SELECT * FROM Loan;
```

I/O | AI Agent | 238 ms

STDIN

Input for the program ( Optional )

Output:

```
+----------------+------------+------------------+----------+---------------------
| Transaction_ID | Account_No | Transaction_Type | Amount   | Transaction_Date
+----------------+------------+------------------+----------+---------------------
|              1 |      10001 | DEPOSIT          | 10000.00 | 2026-08-13 09:04:
|              2 |      10002 | DEPOSIT          | 15000.00 | 2026-08-13 09:04:
|              3 |      10003 | WITHDRAW         | 20000.00 | 2026-08-13 09:04:
|              4 |      10004 | DEPOSIT          |  5000.00 | 2026-08-13 09:04:
|              5 |      10005 | WITHDRAW         | 10000.00 | 2026-08-13 09:04:
+----------------+------------+------------------+----------+---------------------
+---------+-------------+---------------+-------------+---------------+
| Loan_ID | Customer_ID | Loan_Type     | Loan_Amount | Interest_Rate |
+---------+-------------+---------------+-------------+---------------+
|     501 |         101 | Home Loan     | 5000000.00 |          7.50 |
|     502 |         102 | Education Loan | 1000000.00 |          6.50 |
|     503 |         103 | Car Loan      |  800000.00 |          8.20 |
|     504 |         104 | Personal Loan |  500000.00 |         10.50 |
+---------+-------------+---------------+-------------+---------------+
```

Dark | Wiki | MySQL

BankDB.sql

```
DELIMITER //

CREATE PROCEDURE GetAllCustomers()
BEGIN
    SELECT * FROM Customer;
END //

DELIMITER ;

CALL GetAllCustomers();
```

I/O | AI Agent — 156 ms

STDIN

Input for the program ( Optional )

Output:

| Customer_ID | Customer_Name | Phone | Email | City |
|---|---|---|---|---|
| 101 | Ravi Kumar | 9876543210 | ravi@gmail.com | Hyderabad |
| 102 | Priya Sharma | 9876543211 | priya@gmail.com | Vijayawada |
| 103 | Arjun Reddy | 9876543212 | arjun@gmail.com | Bangalore |
| 104 | Sneha Rao | 9876543213 | sneha@gmail.com | Chennai |
| 105 | Kiran Kumar | 9876543214 | kiran@gmail.com | Hyderabad |

Dark | Wiki | MySQL

BankDB.sql

```
DELIMITER //

CREATE PROCEDURE GetAccountDetails(
    IN p_Account_No INT
)
BEGIN
    SELECT *
    FROM Account
    WHERE Account_No = p_Account_No;
END //

DELIMITER ;

CALL GetAccountDetails(10001);
```

I/O | AI Agent — 160 ms

STDIN

Input for the program ( Optional )

Output:

| Account_No | Customer_ID | Account_Type | Balance | Branch |
|---|---|---|---|---|
| 10001 | 101 | Savings | 50000.00 | Hyderabad |

Dark | Wiki | MySQL

BankDB.sql

```
DELIMITER //

CREATE PROCEDURE GetCustomerAccounts(
    IN p_Customer_ID INT
)
BEGIN
    SELECT
        C.Customer_ID,
        C.Customer_Name,
        A.Account_No,
        A.Account_Type,
        A.Balance,
        A.Branch
    FROM Customer C
    JOIN Account A
        ON C.Customer_ID = A.Customer_ID
    WHERE C.Customer_ID = p_Customer_ID;
END //

DELIMITER ;

CALL GetCustomerAccounts(101);
```

I/O | AI Agent — 156 ms

STDIN

Input for the program ( Optional )

Output:

| Customer_ID | Customer_Name | Account_No | Account_Type | Balance | Branch |
|---|---|---|---|---|---|
| 101 | Ravi Kumar | 10001 | Savings | 50000.00 | Hyderabad |

Dark | Wiki | MySQL

BankDB.sql

```sql

DELIMITER //

CREATE PROCEDURE DepositMoney(
    IN p_Account_No INT,
    IN p_Amount DECIMAL(12,2)
)
BEGIN
    UPDATE Account
    SET Balance = Balance + p_Amount
    WHERE Account_No = p_Account_No;
END //

DELIMITER ;

CALL DepositMoney(10001, 5000);

SELECT *
FROM Account
WHERE Account_No = 10001;
```

I/O | AI Agent — 225 ms

STDIN

Input for the program ( Optional )

Output:

```
+------------+-------------+--------------+----------+-----------+
| Account_No | Customer_ID | Account_Type | Balance  | Branch    |
+------------+-------------+--------------+----------+-----------+
|      10001 |         101 | Savings      | 55000.00 | Hyderabad |
+------------+-------------+--------------+----------+-----------+
```

Dark | Wiki | MySQL

BankDB.sql

```sql


DELIMITER //

CREATE PROCEDURE WithdrawMoney(
    IN p_Account_No INT,
    IN p_Amount DECIMAL(12,2)
)
BEGIN
    UPDATE Account
    SET Balance = Balance - p_Amount
    WHERE Account_No = p_Account_No;
END //

DELIMITER ;

CALL WithdrawMoney(10001, 3000);

SELECT *
FROM Account
WHERE Account_No = 10001;
```

I/O | AI Agent — 149 ms

STDIN

Input for the program ( Optional )

Output:

```
+------------+-------------+--------------+----------+-----------+
| Account_No | Customer_ID | Account_Type | Balance  | Branch    |
+------------+-------------+--------------+----------+-----------+
|      10001 |         101 | Savings      | 47000.00 | Hyderabad |
+------------+-------------+--------------+----------+-----------+
```

Dark | Wiki | MySQL

BankDB.sql

```sql
DELIMITER //

CREATE TRIGGER CheckBalance
BEFORE UPDATE ON Account
FOR EACH ROW
BEGIN
    IF NEW.Balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Transaction failed: Insufficient balance';
    END IF;
END //

DELIMITER ;

UPDATE Account
SET Balance = Balance - 60000
WHERE Account_No = 10001;
```

I/O | AI Agent — Error

STDIN

Input for the program ( Optional )

Output:

```
ERROR 1644 (45000) at line 92: Transaction failed: Insufficient balance
```

Dark | Wiki | MySQL

BankDB.sql

```
1
2
3  DELIMITER //
4
5  CREATE TRIGGER CheckTransactionAmount
6  BEFORE INSERT ON Bank_Transaction
7  FOR EACH ROW
8  BEGIN
9      IF NEW.Amount <= 0 THEN
10         SIGNAL SQLSTATE '45000'
11         SET MESSAGE_TEXT =
12         'Transaction amount must be greater than zero';
13     END IF;
14 END //
15
16 DELIMITER ;
17
18 INSERT INTO Bank_Transaction
19 (Account_No, Transaction_Type, Amount)
20 VALUES
21 (10001, 'DEPOSIT', -5000);
```

I/O | AI Agent | Error

STDIN

Input for the program ( Optional )

Output:

```
ERROR 1644 (45000) at line 92: Transaction amount must be greater than zero
```

ady | Dark | Wiki | MySQL

BankDB.sql

```
15 CREATE TRIGGER TransactionAudit
16 AFTER INSERT ON Bank_Transaction
17 FOR EACH ROW
18 BEGIN
19     INSERT INTO Transaction_Audit
20     (
21         Transaction_ID,
22         Account_No,
23         Transaction_Type,
24         Amount
25     )
26     VALUES
27     (
28         NEW.Transaction_ID,
29         NEW.Account_No,
30         NEW.Transaction_Type,
31         NEW.Amount
32     );
33 END //
34
35 DELIMITER ;
36
37 INSERT INTO Bank_Transaction
38 (Account_No, Transaction_Type, Amount)
39 VALUES
40 (10001, 'DEPOSIT', 2500);
41
42 SELECT * FROM Transaction_Audit;
```

I/O | AI Agent | 209 ms

STDIN

Input for the program ( Optional )

Output:

```
+----------+----------------+------------+------------------+---------+----------
| Audit_ID | Transaction_ID | Account_No | Transaction_Type | Amount  | Audit_Da
+----------+----------------+------------+------------------+---------+----------
|        1 |              6 |      10001 | DEPOSIT          | 2500.00 | 2026-08-1
+----------+----------------+------------+------------------+---------+----------
```

ady | Dark | Wiki | MySQL

BankDB.sql

```
5  AFTER INSERT ON Bank_Transaction
6  FOR EACH ROW
7  BEGIN
8      IF NEW.Transaction_Type = 'DEPOSIT' THEN
9
10         UPDATE Account
11         SET Balance = Balance + NEW.Amount
12         WHERE Account_No = NEW.Account_No;
13
14     ELSEIF NEW.Transaction_Type = 'WITHDRAW' THEN
15
16         UPDATE Account
17         SET Balance = Balance - NEW.Amount
18         WHERE Account_No = NEW.Account_No;
19
20     END IF;
21 END //
22
23 DELIMITER ;
24
25 INSERT INTO Bank_Transaction
26 (Account_No, Transaction_Type, Amount)
27 VALUES
28 (10001, 'DEPOSIT', 5000);
29
30 SELECT *
31 FROM Account
32 WHERE Account_No = 10001;
```

I/O | AI Agent | 292 ms

STDIN

Input for the program ( Optional )

Output:

```
+------------+-------------+--------------+----------+-----------+
| Account_No | Customer_ID | Account_Type | Balance  | Branch    |
+------------+-------------+--------------+----------+-----------+
|      10001 |         101 | Savings      | 55000.00 | Hyderabad |
+------------+-------------+--------------+----------+-----------+
```

ady | Dark | Wiki | MySQL

BankDB.sql

```
CREATE TRIGGER PreventInsufficientWithdrawal
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    DECLARE CurrentBalance DECIMAL(12,2);

    SELECT Balance
    INTO CurrentBalance
    FROM Account
    WHERE Account_No = NEW.Account_No;

    IF NEW.Transaction_Type = 'WITHDRAW'
       AND NEW.Amount > CurrentBalance THEN

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Withdrawal failed: Insufficient balance';

    END IF;
END //

DELIMITER ;

INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'WITHDRAW', 1000000);
```

I/O | AI Agent | Error

STDIN

Input for the program ( Optional )

Output:

```
ERROR 1644 (45000) at line 169: Withdrawal failed: Insufficient balance
```

Dark | Wiki | MySQL

BankDB.sql

```
BEGIN
    WHERE Account_No = SenderAccount;

    IF SenderBalance < TransferAmount THEN

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Transfer failed: Insufficient balance';

    ELSE

        UPDATE Account
        SET Balance = Balance - TransferAmount
        WHERE Account_No = SenderAccount;

        UPDATE Account
        SET Balance = Balance + TransferAmount
        WHERE Account_No = ReceiverAccount;

    END IF;
END //

DELIMITER ;

CALL TransferMoney(10001, 10002, 5000);
SELECT *
FROM Account
WHERE Account_No IN (10001, 10002);
```

I/O | AI Agent | 141 ms

STDIN

Input for the program ( Optional )

Output:

| Account_No | Customer_ID | Account_Type | Balance | Branch |
|---|---|---|---|---|
| 10001 | 101 | Savings | 50000.00 | Hyderabad |
| 10002 | 102 | Savings | 80000.00 | Vijayawada |

Dark | Wiki | MySQL

BankDB.sql

```

DELIMITER //

CREATE PROCEDURE GetCustomerLoans(
    IN p_Customer_ID INT
)
BEGIN
    SELECT
        C.Customer_Name,
        L.Loan_ID,
        L.Loan_Type,
        L.Loan_Amount,
        L.Interest_Rate
    FROM Customer C
    JOIN Loan L
        ON C.Customer_ID = L.Customer_ID
    WHERE C.Customer_ID = p_Customer_ID;
END //

DELIMITER ;

CALL GetCustomerLoans(101);
```

I/O | AI Agent | 233 ms

STDIN

Input for the program ( Optional )

Output:

| Customer_Name | Loan_ID | Loan_Type | Loan_Amount | Interest_Rate |
|---|---|---|---|---|
| Ravi Kumar | 501 | Home Loan | 5000000.00 | 7.50 |

Dark | Wiki | MySQL

BankDB.sql

```
DELIMITER //

CREATE PROCEDURE HighBalanceAccounts(
    IN MinimumBalance DECIMAL(12,2)
)
BEGIN
    SELECT *
    FROM Account
    WHERE Balance >= MinimumBalance
    ORDER BY Balance DESC;
END //

DELIMITER ;

CALL HighBalanceAccounts(50000);
```

I/O | AI Agent | 691 ms

STDIN

Input for the program ( Optional )

Output:

| Account_No | Customer_ID | Account_Type | Balance | Branch |
|---|---|---|---|---|
| 10003 | 103 | Current | 120000.00 | Bangalore |
| 10005 | 105 | Current | 90000.00 | Hyderabad |
| 10002 | 102 | Savings | 75000.00 | Vijayawada |
| 10001 | 101 | Savings | 55000.00 | Hyderabad |

Ready | Dark | Wiki | MySQL

BankDB.sql

```
)
BEGIN
    SELECT *
    FROM Account
    WHERE Balance >= MinimumBalance
    ORDER BY Balance DESC;
END //

DELIMITER ;

DELIMITER //

CREATE PROCEDURE GetBalance(
    IN p_Account_No INT,
    OUT p_Balance DECIMAL(12,2)
)
BEGIN
    SELECT Balance
    INTO p_Balance
    FROM Account
    WHERE Account_No = p_Account_No;
END //

DELIMITER ;

CALL GetBalance(10001, @CurrentBalance);

SELECT @CurrentBalance;
```

I/O | AI Agent | 191 ms

STDIN

Input for the program ( Optional )

Output:

| @CurrentBalance |
|---|
| 55000.00 |

BankDB.sql

```
CREATE PROCEDURE GetBalance(
    OUT p_Balance DECIMAL(12,2)
)
BEGIN
    SELECT Balance
    INTO p_Balance
    FROM Account
    WHERE Account_No = p_Account_No;
END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER CheckBalance
BEFORE UPDATE ON Account
FOR EACH ROW
BEGIN
    IF NEW.Balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Transaction failed: Insufficient balance';
    END IF;
END //

DELIMITER ;

SHOW TRIGGERS;
```

I/O | AI Agent | 259 ms

STDIN

Input for the program ( Optional )

Output:

| Trigger | Event | Table | Statement |
|---|---|---|---|
| CheckBalance | UPDATE | account | BEGIN<br>IF NEW.Balance < 0 THEN<br>SIGNAL SQLSTATE '45000'<br>SET MESSAGE_TEXT =<br>'Transaction failed: Insufficient balance';<br>END IF;<br>END |
| PreventInsufficientWithdrawal | INSERT | bank_transaction | BEGIN<br>DECLARE CurrentBalance DECIMAL(12,2);<br><br>SELECT Balance<br>INTO CurrentBalance<br>FROM Account<br>WHERE Account_No = NEW.Account_No;<br><br>IF NEW.Transaction_Type = 'WITHDRAW' |

Ready | Dark | Wiki | MySQL

BankDB.sql

```
CREATE TRIGGER CheckBalance
BEFORE UPDATE ON Account
FOR EACH ROW
BEGIN
    IF NEW.Balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Transaction failed: Insufficient balance';
    END IF;
END //

DELIMITER ;

DELIMITER //

CREATE PROCEDURE GetBranchAccounts(
    IN p_Branch VARCHAR(50)
)
BEGIN
    SELECT *
    FROM Account
    WHERE Branch = p_Branch;
END //

DELIMITER ;

CALL GetBranchAccounts('Hyderabad');
```

I/O | AI Agent | 181 ms

STDIN

Input for the program ( Optional )

Output:

| Account_No | Customer_ID | Account_Type | Balance | Branch |
|---|---|---|---|---|
| 10001 | 101 | Savings | 55000.00 | Hyderabad |
| 10005 | 105 | Current | 90000.00 | Hyderabad |

Dark | Wiki | MySQL

BankDB.sql

```
BEGIN
    WHERE Branch = p_Branch;
END //

DELIMITER ;

DELIMITER //

CREATE PROCEDURE GetCustomerFullDetails(
    IN p_Customer_ID INT
)
BEGIN
    SELECT
        C.Customer_Name,
        C.Phone,
        C.Email,
        A.Account_No,
        A.Account_Type,
        A.Balance
    FROM Customer C
    JOIN Account A
        ON C.Customer_ID = A.Customer_ID
    WHERE C.Customer_ID = p_Customer_ID;
END //

DELIMITER ;

CALL GetCustomerFullDetails(101);
```

I/O | AI Agent | 189 ms

STDIN

Input for the program ( Optional )

Output:

| Customer_Name | Phone | Email | Account_No | Account_Type | Balar |
|---|---|---|---|---|---|
| Ravi Kumar | 9876543210 | ravi@gmail.com | 10001 | Savings | 5500 |

Dark | Wiki | MySQL

BankDB.sql

```
BEGIN
    SELECT
        A.Balance
    FROM Customer C
    JOIN Account A
        ON C.Customer_ID = A.Customer_ID
    WHERE C.Customer_ID = p_Customer_ID;
END //

DELIMITER ;

DELIMITER //

CREATE PROCEDURE GetTotalCustomerBalance(
    IN p_Customer_ID INT
)
BEGIN
    SELECT
        Customer_ID,
        SUM(Balance) AS Total_Balance
    FROM Account
    WHERE Customer_ID = p_Customer_ID
    GROUP BY Customer_ID;
END //

DELIMITER ;

CALL GetTotalCustomerBalance(101);
```

I/O | AI Agent | 279 ms

STDIN

Input for the program ( Optional )

Output:

```
+-------------+---------------+
| Customer_ID | Total_Balance |
+-------------+---------------+
|         101 |      55000.00 |
+-------------+---------------+
```

Ready | Dark | Wiki | MySQL

BankDB.sql

```
BEGIN
    SELECT
        A.Balance
    FROM Customer C
    JOIN Account A
        ON C.Customer_ID = A.Customer_ID
    WHERE C.Customer_ID = p_Customer_ID;
END //

DELIMITER ;

DELIMITER //

CREATE PROCEDURE GetTotalCustomerBalance(
    IN p_Customer_ID INT
)
BEGIN
    SELECT
        Customer_ID,
        SUM(Balance) AS Total_Balance
    FROM Account
    WHERE Customer_ID = p_Customer_ID
    GROUP BY Customer_ID;
END //

DELIMITER ;

CALL HighBalanceAccounts(100000);
```

I/O | AI Agent | 189 ms

STDIN

Input for the program ( Optional )

Output:

```
+------------+-------------+--------------+-----------+-----------+
| Account_No | Customer_ID | Account_Type | Balance   | Branch    |
+------------+-------------+--------------+-----------+-----------+
|      10003 |         103 | Current      | 120000.00 | Bangalore |
+------------+-------------+--------------+-----------+-----------+
```

Ready | Dark | Wiki | MySQL